

Requirements of an algorithm?



1. Finiteness

- terminates after a finite number of steps

2. Definiteness

- Predictable, clarity and unambiguously specified

3. Clearly specified input

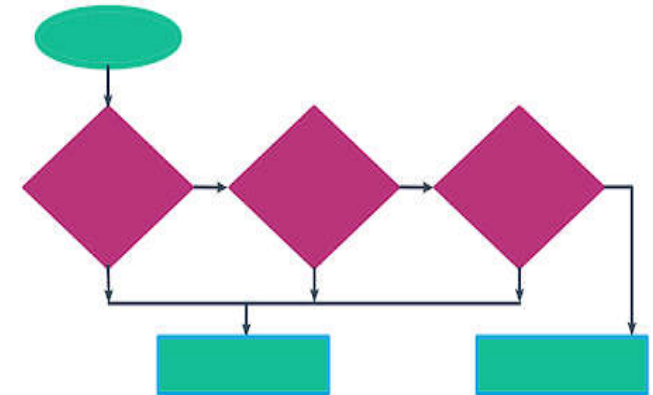
- valid inputs are clearly specified

4. Clearly specified/expected output

- produce correct output from valid input

5. Effectiveness

- steps are sufficiently simple and basic

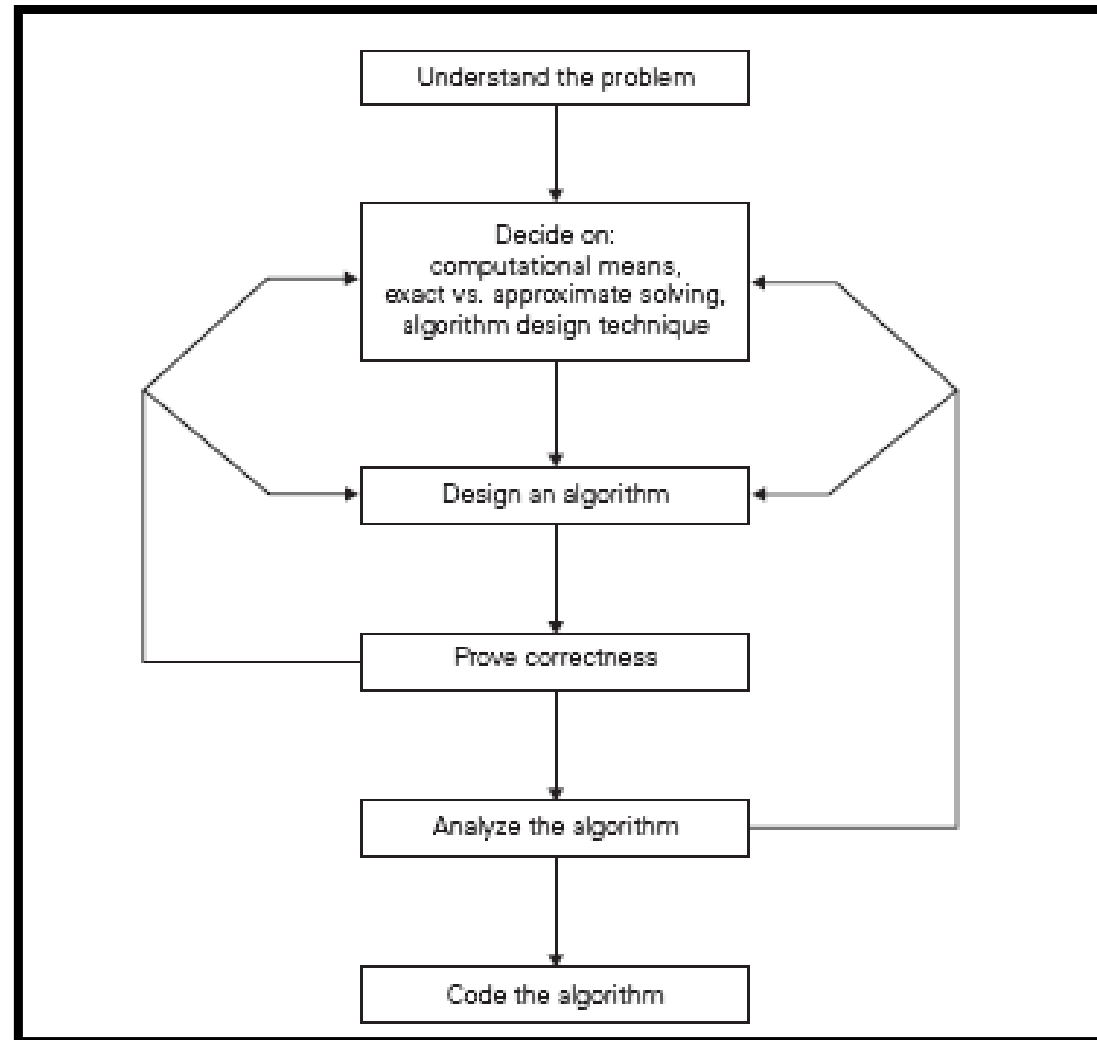


Fundamentals of Algorithmic Problem Solving



L
P
U

- Recipe
- Process
- Method
- Technique
- Procedure
- Routine.



Some Well-known Computational Problems

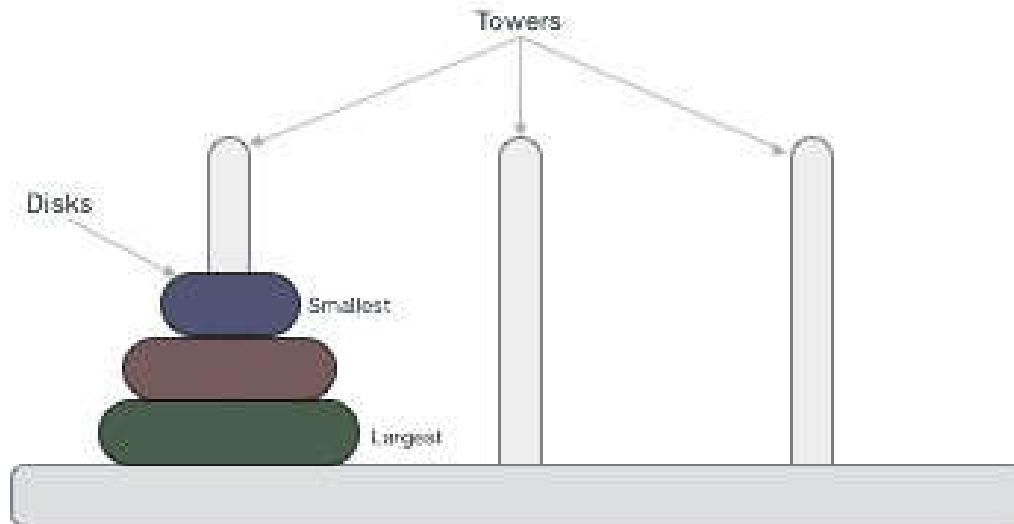
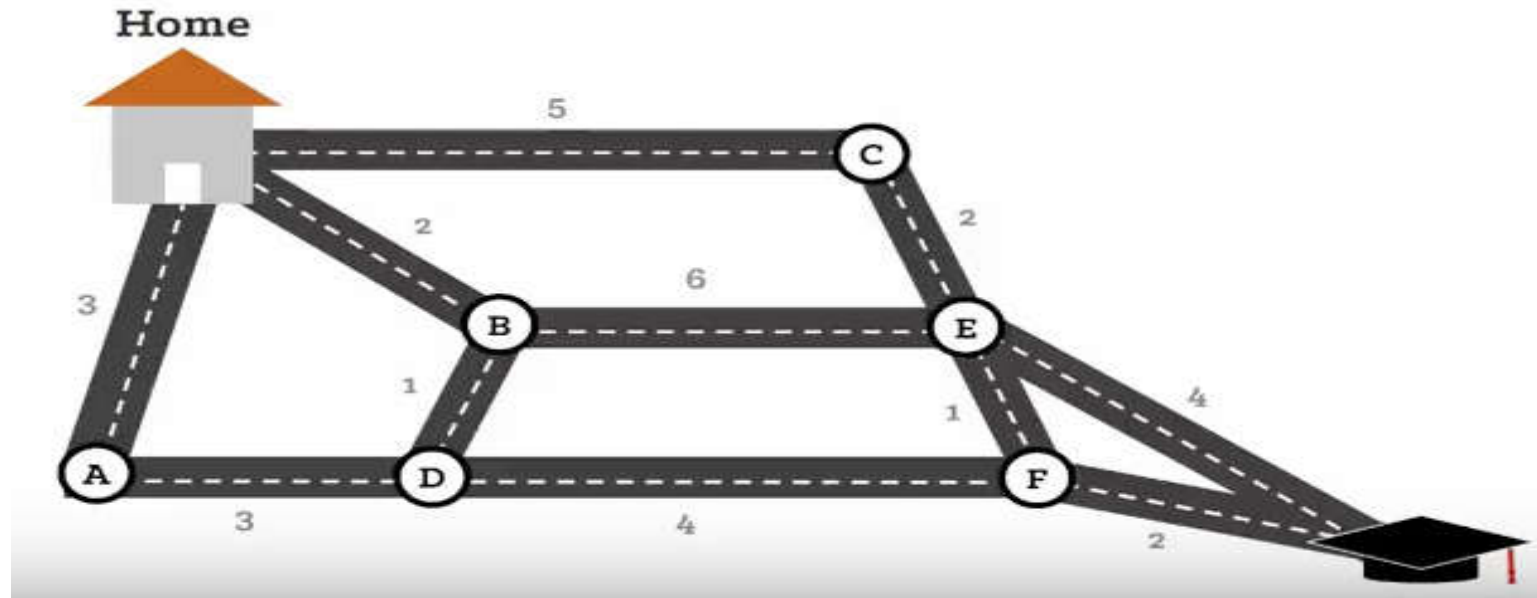


Few Examples are:

Sorting	Traveling salesman problem
Searching	Knapsack problem
Shortest paths in a graph	Chess
Minimum cost spanning tree	Towers of Hanoi
Primality testing	Program termination

Some of these problems don't have efficient algorithms, or algorithms at all!

Minimum cost spanning tree



Example of computational problem: sorting



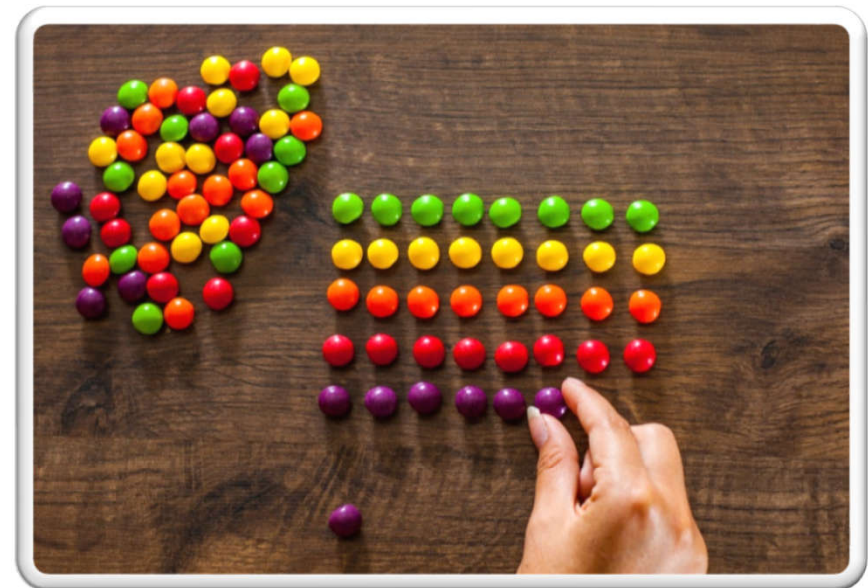
□ Statement of problem:

- *Input:* A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
- *Output:* reordering so that $a'_i \leq a'_j$ whenever $i < j$

□ Instance: The sequence $\langle 5, 3, 2, 8, 3 \rangle$

□ Algorithms:

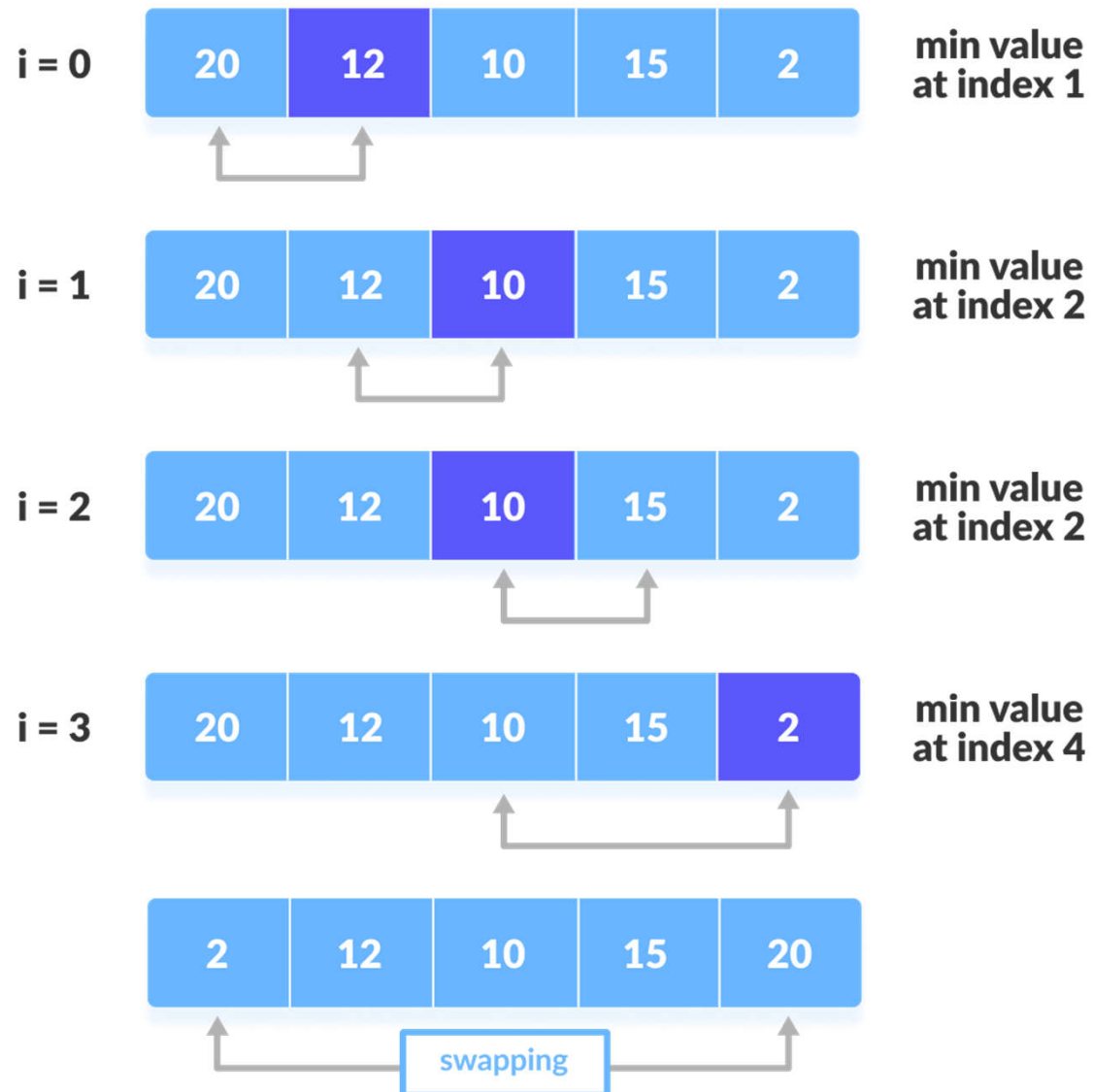
- Selection sort
- Insertion sort
- Merge sort
- (many others)



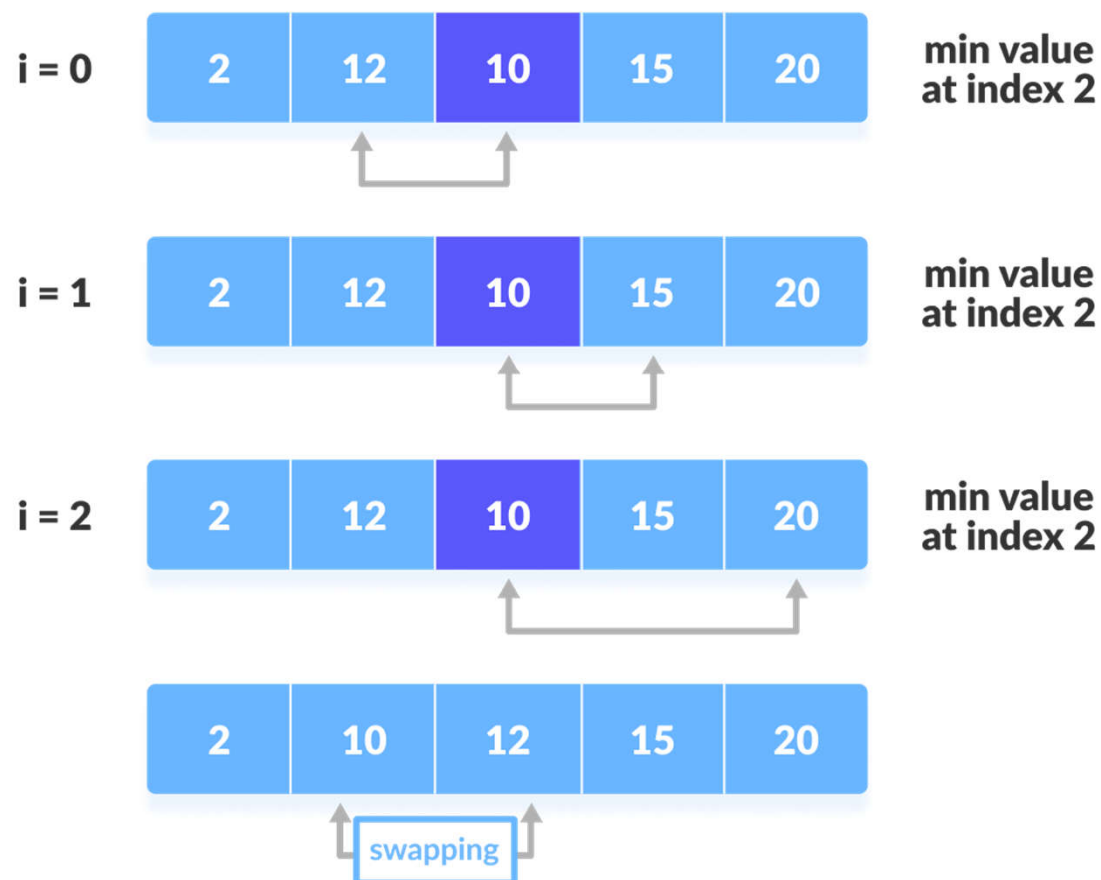
Selection Sort



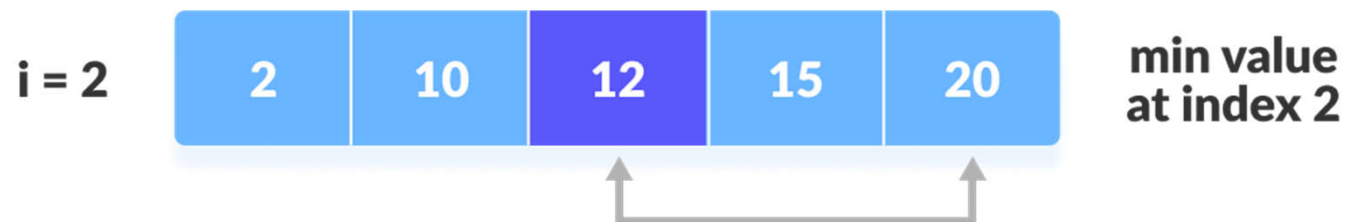
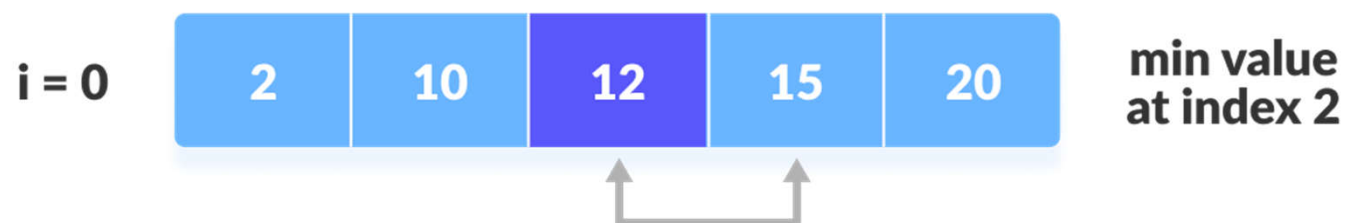
step = 0



step = 1



step = 2



step = 3

i = 0



**min value
at index 3**



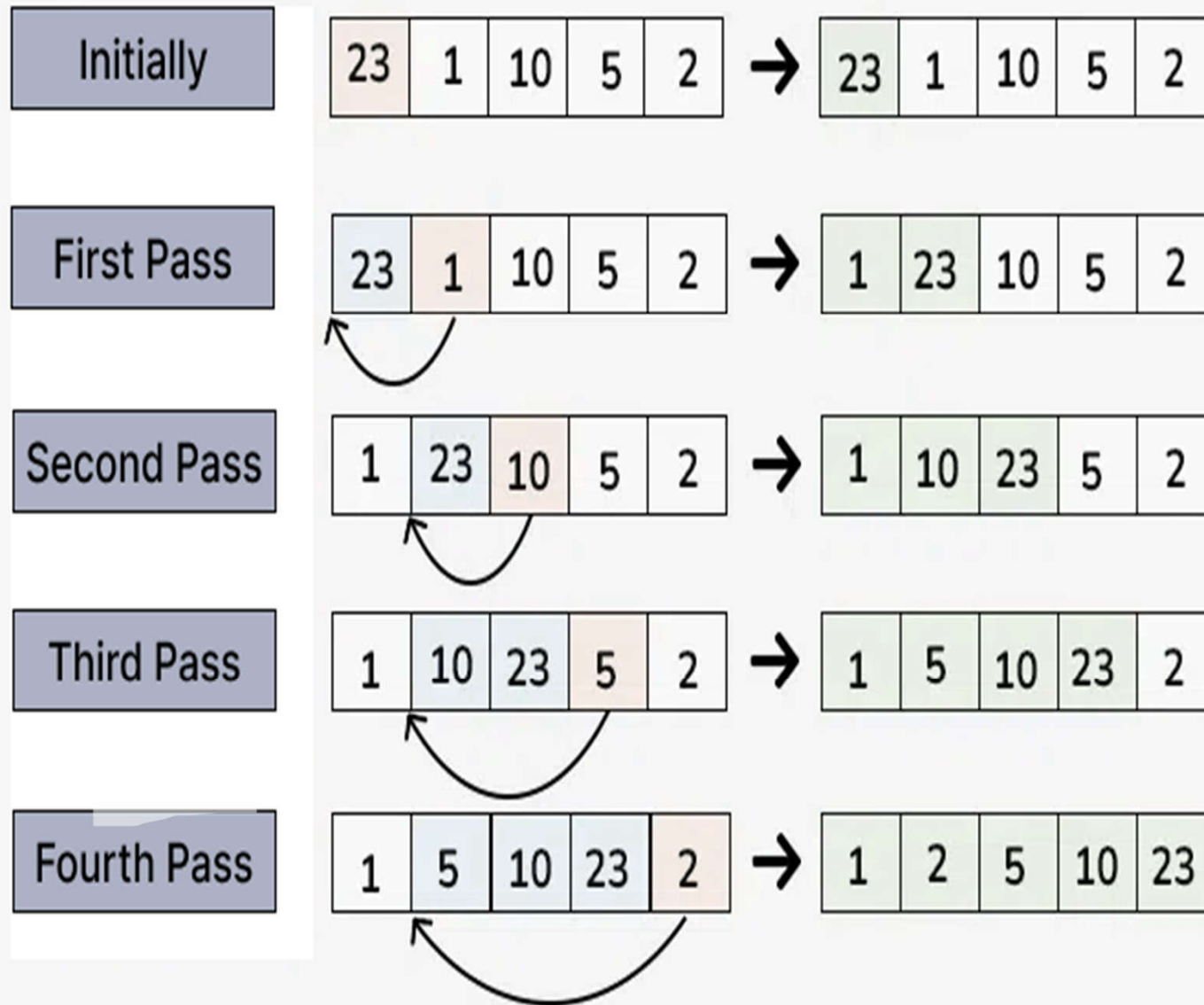
already in place

Insertion Sort



23	1	10	5	2
----	---	----	---	---

Insertion Sort

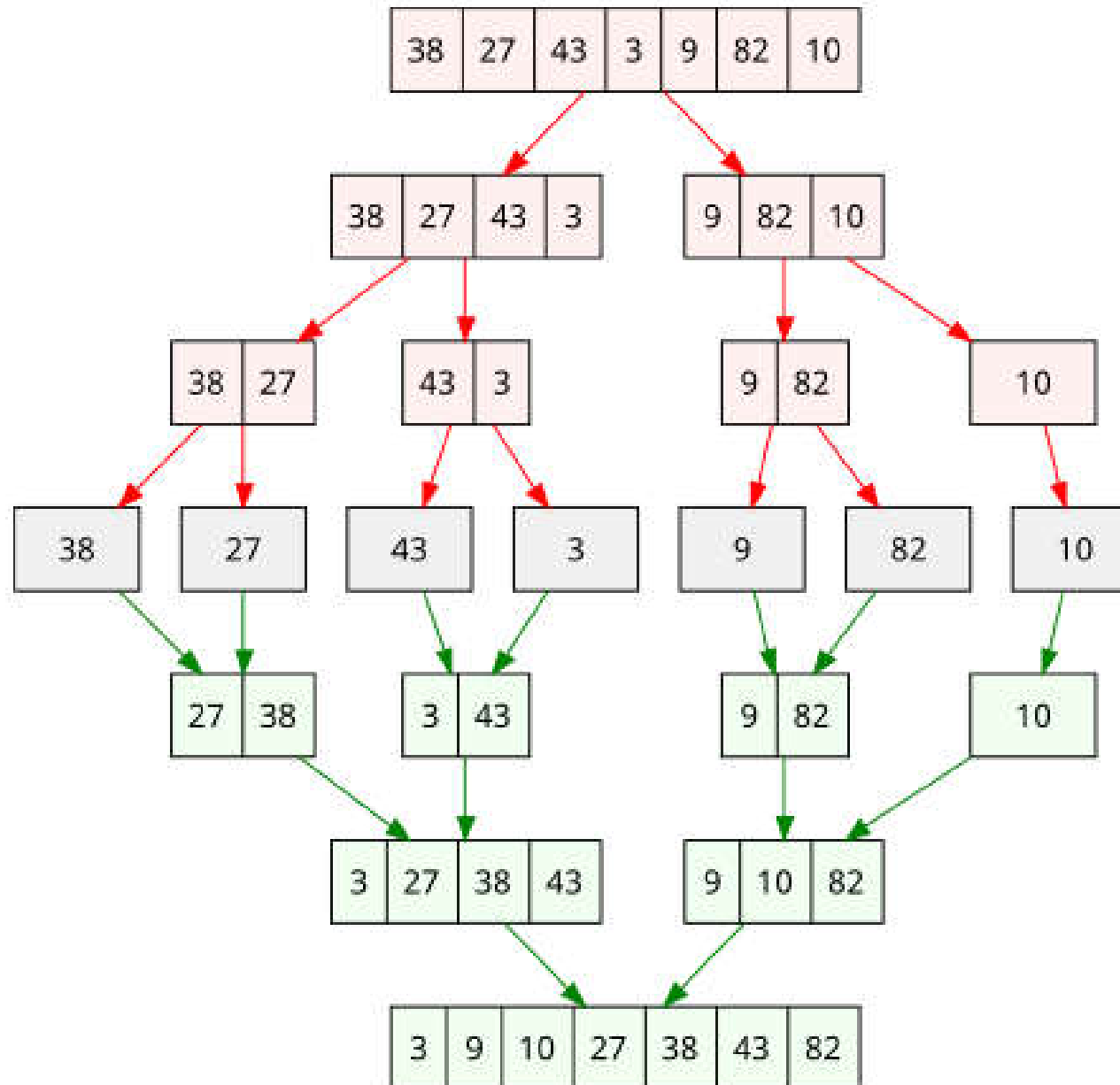


Merge Sort



38	27	43	3	9	82	10
----	----	----	---	---	----	----

Merge Sort



Bubble Sort

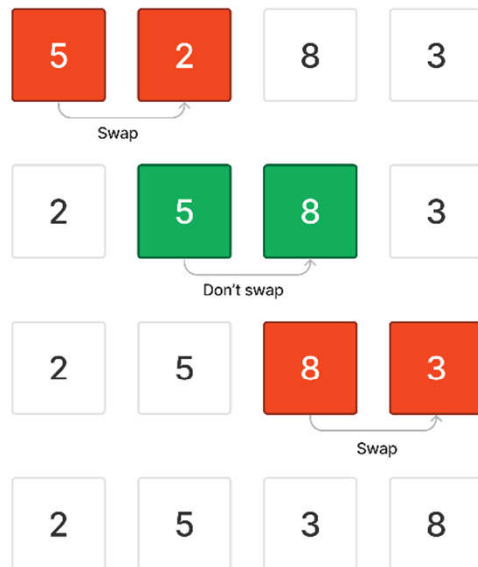


[5 2 8 3]

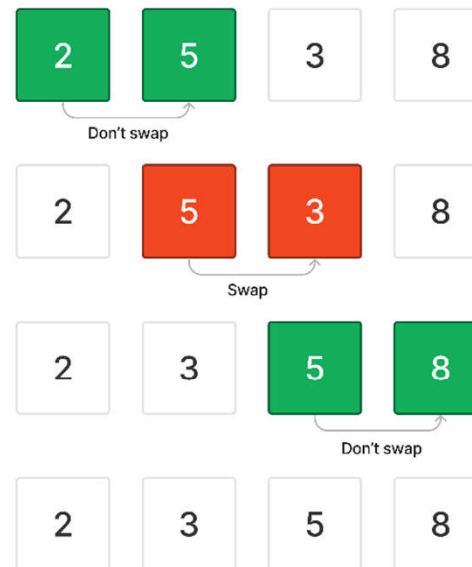
Bubble Sort



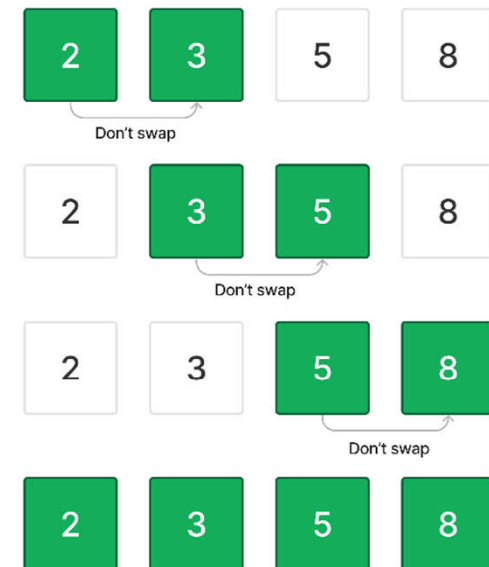
First pass



Second pass



Third pass



Basic Issues Related to Algorithms



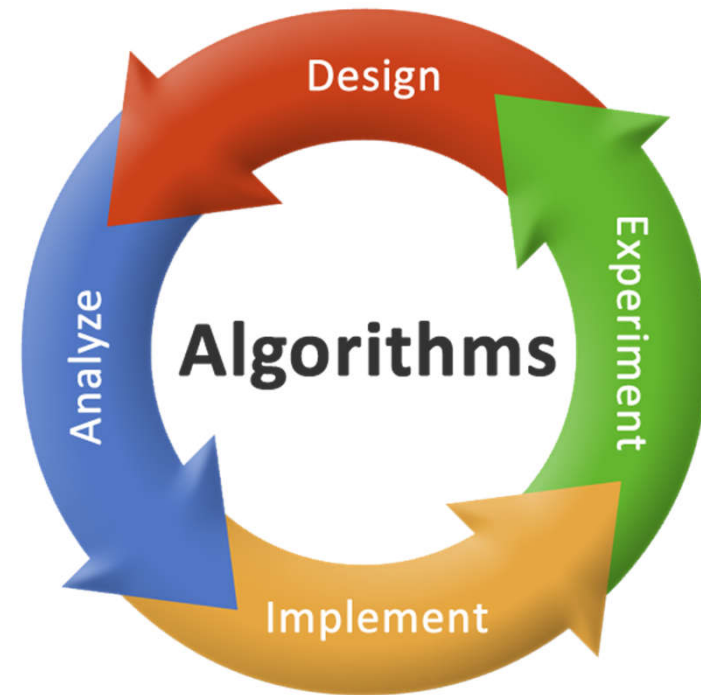
- How to design algorithms
- How to express algorithms
- Proving correctness
- Efficiency (or complexity) analysis
 - Theoretical analysis
 - Empirical analysis
- Optimality



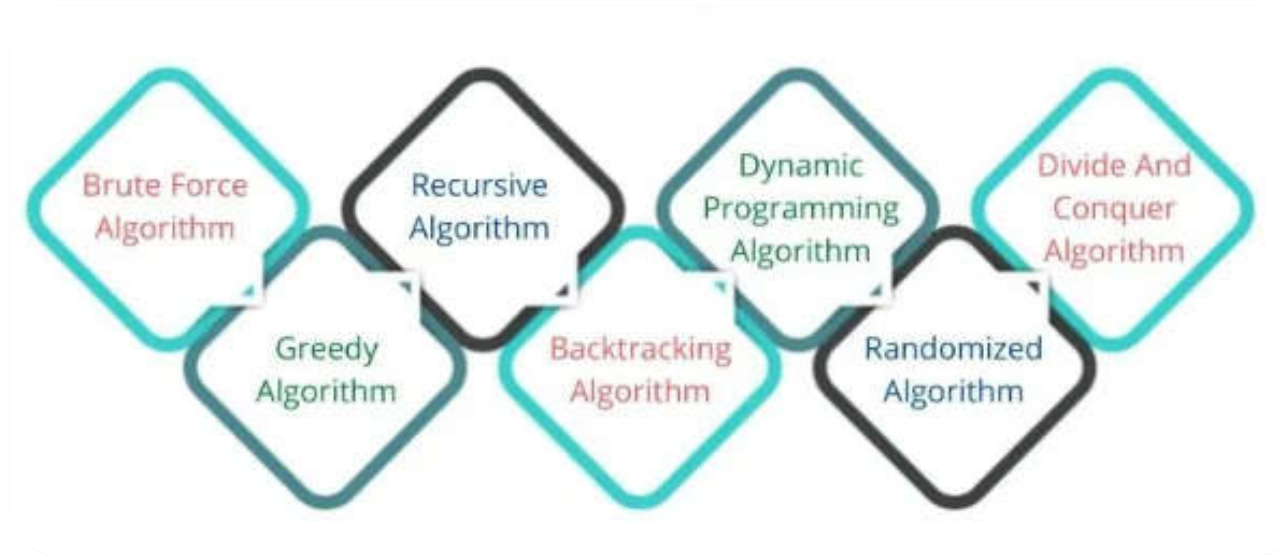
Two main issues related to algorithms



- How to design algorithms
- How to analyze algorithm efficiency



Algorithm design strategies

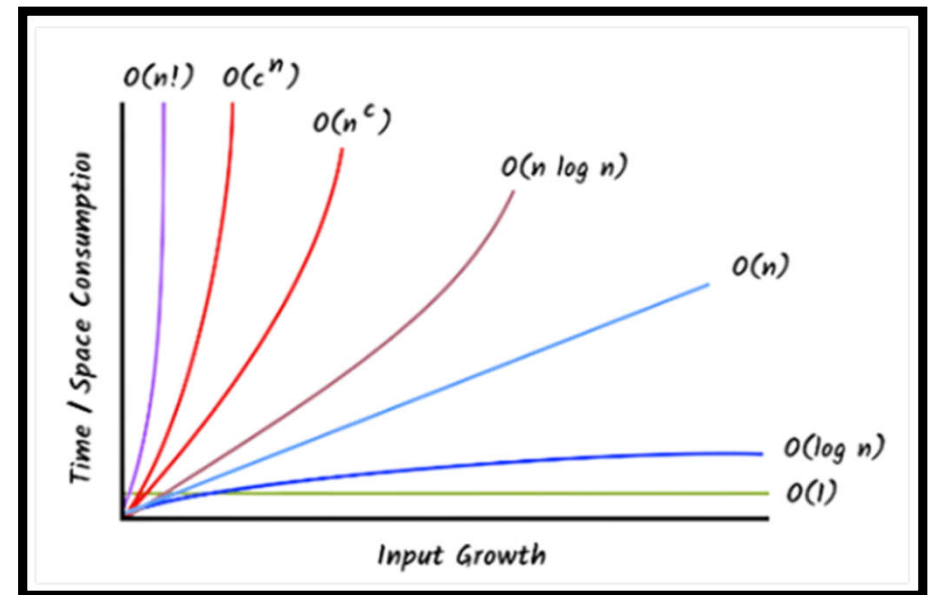


- Brute force
- Greedy approach
- Divide and conquer
- Dynamic programming
- Decrease and conquer
- Backtracking and branch-and-bound
- Transform and conquer
- Space and time tradeoffs

Analysis of Algorithms



- How good is the algorithm?
 - Correctness
 - Time efficiency
 - Space efficiency
- Does there exist a better algorithm?
 - Lower bounds
 - Optimality



Why study algorithms?



- **Theoretical Importance**
 - Core of computer science
- **Practical Importance**
 - Toolkit of known algorithms
- **Design Framework**
 - Solves unique, new problems
 - Provides structured approaches
- **Efficiency Analysis**
 - Measures performance
 - Optimizes time and resources



Example: Google's PageRank Technology